12 OPERATOR OVERLOADING • DETAILED SOLUTIONS

programming solutions

Detailed Theory, Exact C++ Rules, Memory Models & Exam Reference

Detailed Solutions — Programming Questions (Module 12:

Operator Overloading)

Solution to Q11 (`complex_system` with `+`, `-`, `*`, and `==`)

CPP

```cpp
#include <iostream>
using namespace std;

class complex_system {
private:
    int real;
    int imag;

public:
    complex_system(int r = 0, int i = 0) {
        real = r;
        imag = i;
    }

    complex_system operator+(const complex_system& obj) const {
        return complex_system(real + obj.real, imag + obj.imag);
    }

    complex_system operator-(const complex_system& obj) const {
        return complex_system(real - obj.real, imag - obj.imag);
    }

    complex_system operator*(const complex_system& obj) const {
        int r = (real * obj.real) - (imag * obj.imag);
        int i = (real * obj.imag) + (imag * obj.real);
        return complex_system(r, i);
    }

    bool operator==(const complex_system& obj) const {
        return (real == obj.real) && (imag == obj.imag);
    }

    void Show_Num() const {
        cout << real << " + " << imag << "i" << endl;
    }
};

int main() {
    complex_system c1(3, 2);
    complex_system c2(1, 4);

    complex_system sum = c1 + c2;
    complex_system diff = c1 - c2;
    complex_system prod = c1 * c2; // (3*1 - 2*4) + (3*4 + 2*1)i = -5 + 14i

    sum.Show_Num();
    diff.Show_Num();
    prod.Show_Num();
    cout << "Equal? " << (c1 == c2) << endl;
```

```
    return 0;
}
/*
Expected Output:
4 + 6i
2 + -2i
-5 + 14i
Equal? 0
*/
```

Solution to Q12 (`distance_measure` with Normalized `+` and `<`)

CPP

```cpp
#include <iostream>
using namespace std;

class distance_measure {
private:
    int feet;
    int inches;

public:
    distance_measure(int f = 0, int in = 0) {
        feet = f + (in / 12);
        inches = in % 12;
    }

    distance_measure operator+(const distance_measure& d) const {
        return distance_measure(feet + d.feet, inches + d.inches);
    }

    bool operator<(const distance_measure& d) const {
        int total1 = (feet * 12) + inches;
        int total2 = (d.feet * 12) + d.inches;
        return total1 < total2;
    }

    void display() const {
        cout << feet << " ft " << inches << " in" << endl;
    }
};

int main() {
    distance_measure d1(5, 9);
    distance_measure d2(3, 8);
    distance_measure d3 = d1 + d2; // 8 ft 17 in → normalized to 9 ft 5 in

    d3.display();
    cout << "Is d2 < d1? " << (d2 < d1) << endl;
    return 0;
}
/*
Expected Output:
9 ft 5 in
Is d2 < d1? 1
*/
```